

CS 577 Deep Learning

Homework 2: Inside an MLP

Irma Modzgyrshvili (Hawk username: A20597307)

Fall 2026

Part 0: Conventions and Reproducible Setup

0.1 Environment and public tests

Environment Item	Version / Value
Python	3.13.5
Platform	macOS-15.7.4-arm64-arm-64bit-Mach-O
NumPy	2.5.2
PyTorch	2.13.0
Required device	CPU

0.2 Shape and reduction contract

The targets are stored with shape $(B, 1)$ so they match the logits shape $(B, 1)$. If we combine an array with shape $(B, 1)$ and another with shape $(B,)$, NumPy broadcasting can produce an incorrect result with shape (B, B) . Because the loss is the mean over the batch,

$$J = \frac{1}{B} \sum_{i=1}^B \ell_i,$$

the factor $\frac{1}{B}$ enters backward propagation in

$$\frac{\partial J}{\partial S} = \frac{\sigma(S) - y}{B}.$$

Part 1: Forward and Backward Mathematics

1.1 Forward shape trace

Quantity	Expression / role	Shape
X	input batch	(B, D)
W_1, b_1	first affine parameters	$(D, H), (H,)$
Z_1	$XW_1 + b_1$	(B, H)
A_1	$\text{ReLU}(Z_1)$	(B, H)
W_2, b_2	output affine parameters	$(H, 1), (1,)$
S	logits	$(B, 1)$
y	binary targets	$(B, 1)$
J	mean batch loss	scalar

The first bias b_1 has shape $(H,)$ and is broadcast across all B examples when it is added to XW_1 , which has shape (B, H) .

The second bias b_2 has shape $(1,)$ and is broadcast across all B examples when it is added to A_1W_2 , which has shape $(B, 1)$.

The values in S are logits, or raw model outputs. They are not probabilities. The probability of class 1 is obtained using $\sigma(S)$.

Since J is the mean loss over the batch, it is a scalar.

1.2 BCE-with-logits derivative

For one example, let

$$p = \sigma(s).$$

The binary cross-entropy loss is

$$\ell(s, y) = -y \log p - (1 - y) \log(1 - p).$$

First, differentiate with respect to p :

$$\frac{\partial \ell}{\partial p} = -\frac{y}{p} + \frac{1 - y}{1 - p}.$$

Since

$$p = \sigma(s) = \frac{1}{1 + e^{-s}},$$

the sigmoid derivative is

$$\frac{\partial p}{\partial s} = p(1 - p).$$

Using the chain rule,

$$\frac{\partial \ell}{\partial s} = \frac{\partial \ell}{\partial p} \frac{\partial p}{\partial s}.$$

Therefore,

$$\frac{\partial \ell}{\partial s} = \left(-\frac{y}{p} + \frac{1-y}{1-p} \right) p(1-p).$$

Simplifying,

$$= -y(1-p) + (1-y)p$$

$$= -y + yp + p - yp$$

$$= p - y.$$

Since $p = \sigma(s)$,

$$\frac{\partial \ell}{\partial s} = \sigma(s) - y.$$

The full loss is the mean over the batch,

$$J = \frac{1}{B} \sum_{i=1}^B \ell_i,$$

so the first backward signal is

$$\boxed{\frac{\partial J}{\partial S} = \frac{\sigma(S) - y}{B}}.$$

1.3 Complete backward chain

Part 2: NumPy Forward and Backward Implementation

2.1 Stable primitives

2.2 Fixed-case forward pass

2.3 Fixed-case manual backward pass

Parameter	Gradient shape	Gradient norm
W_1	TODO	TODO
b_1	TODO	TODO
W_2	TODO	TODO
b_2	TODO	TODO

Part 3: Two Independent Gradient Checks

3.1 Centered finite differences

Parameter	Maximum absolute error	Relative error
W_1	TODO	TODO
b_1	TODO	TODO
W_2	TODO	TODO
b_2	TODO	TODO

3.2 PyTorch autograd comparison

Part 4: Nonlinear Experiment

4.1 Linear baseline

4.2 Manual MLP

4.3 Interpretation

Part 5: Reflection

Most useful cached value. TODO

First gradient-check investigations. TODO

Why close is not bit-identical. TODO

Assistance and Generative-AI Disclosure

TODO